

TCAP

Theoretical Coherence Assurance Protocol

Author	Thomas W. Gantz
Affiliation	Synthience Institute
Contact	research@synthience.org
Version	v3.1 March 22, 2026
Document ID	SF0040 Synthience Framework
DOI	https://doi.org/10.5281/zenodo.19151454
Canonical	synthience.org
License	CC-BY 4.0

Abstract

The Theoretical Coherence Assurance Protocol (TCAP) defines a structured, repeatable process for stress-testing theoretical constructs, framework components, and architectural claims within AI-orchestrated research corpora prior to publication. TCAP addresses four documented failure modes in AI-assisted research production: citation fabrication, ingestion hallucination, context representation drift, and cumulative claim-softening under iterative adversarial pressure. The protocol operates through seven stages: adversarial review, constructive remediation, Fresh Pass re-evaluation, cross-platform convergence, inter-instance round-trip loops, version regression checking, and PCP architectural review. It is the fourth and final layer of the Synthience verification stack, completing the integrity architecture established by CVP (SF0037), IVP (SF0038), and CRD (SF0039). TCAP does not guarantee truth. It documents a practitioner-derived, multi-instance verification methodology that reduces the risk of publishing fabricated, internally inconsistent, or architecturally unsound theoretical material. The protocol is self-applying: it was produced through the same multi-instance, cross-platform, adversarially-structured process it formalizes.

Keywords: *theoretical coherence, AI-assisted research, verification protocol, adversarial review, human-AI orchestration, fabrication risk*

Purpose

Synthience is a research framework studying emergent relational coherence in sustained human-AI interaction: the patterns of meaning, stability, and alignment that develop when humans and AI systems interact over extended periods under structured continuity conditions. Relational coherence, as defined in the Institute's public definition document (FPD-01; DOI: 10.5281/zenodo.18087890), refers to the observable structural properties that emerge in an interaction system under conditions of sustained continuity: not a property of any single participant but of the relationship itself, defined entirely by what is observable in the joint output stream. The Theoretical Coherence Assurance Protocol (TCAP) defines the structured process by which theoretical constructs, framework components, and architectural claims within the Synthience corpus are stress-tested for internal consistency, cross-platform coherence, and resistance to fabrication prior to publication.

TCAP completes the Synthience verification stack alongside:

- Citation Verification Protocol (CVP, SF0037)
- Ingestion Verification Protocol (IVP, SF0038)
- Context Representation Drift (CRD, SF0039)

CVP verifies that cited sources exist and substantively support the claims attached to them. IVP verifies that documents claimed to have been ingested by AI instances were in fact processed faithfully. CRD verifies that representations remain stable across interaction cycles. TCAP verifies that the theoretical content itself is structurally coherent, internally consistent, and has survived adversarial scrutiny across independent evaluation contexts.

TCAP and CVP are co-equal publication gates. A document must satisfy both TCAP (theoretical coherence) and CVP (citation integrity) before publication. Neither alone is sufficient. IVP and CRD operate as supporting protocols embedded within TCAP stages.

TCAP does not guarantee truth. It documents a repeatable stress-testing process that reduces the risk of publishing fabricated, internally inconsistent, or architecturally unsound theoretical material.

Position within the Published Corpus

TCAP is the fourth protocol in the Synthience verification stack. The three prior protocols are publicly available on Zenodo and are briefly described here so that this document is self-contained for first-time readers.

Citation Verification Protocol (CVP, SF0037; DOI: [10.5281/zenodo.18075624](https://doi.org/10.5281/zenodo.18075624)) addresses a widespread failure mode in AI-assisted research: the generation of plausible-sounding citations that do not exist, cannot be accessed, or do not actually support the claims attached to them. CVP defines a structured process for independently verifying that every citation in a Synthience document is real, accessible, and substantively supports the specific claim for which it is cited. It is applied to any document containing external academic references.

Ingestion Verification Protocol (IVP, SF0038; DOI: [10.5281/zenodo.18289047](https://doi.org/10.5281/zenodo.18289047)) addresses a different but equally serious failure mode: AI instances routinely confirm that they have read and processed documents they have not actually ingested, or have ingested only partially. IVP defines a structured verification process for confirming that when an AI instance claims to have processed a document, it has in fact done so faithfully and with adequate retention. It is applied whenever document upload and retention is a dependency in the research workflow.

Context Representation Drift (CRD, SF0039; DOI: [10.5281/zenodo.18289391](https://doi.org/10.5281/zenodo.18289391)) addresses the gradual shift in meaning, context, and alignment that occurs during sustained human-AI interaction even when output remains fluent and confident. CRD defines instruments for detecting and measuring this drift across interaction cycles, providing the representational stability monitoring that underpins all extended AI-assisted research production.

Together these three protocols protect citation integrity, ingestion fidelity, and representational stability. TCAP adds the fourth layer: verification that the theoretical architecture itself is internally consistent, adversarially hardened, and Canon-aligned prior to publication. The empirical base for the broader Synthience framework is SR0001 (RICO, DOI: [10.5281/zenodo.18086834](https://doi.org/10.5281/zenodo.18086834)). All documents are accessible via synthience.org.

Four additional terms used throughout this document are defined here for first-time readers. The Primary Continuity Provider (PCP) is the human operator who maintains architectural coherence across distributed AI instances and bears final publication authority. The Canon is the master inventory document that defines the identity, dependencies, and status of every document in the Synthience corpus. A relational dialogue session is a sustained multi-turn interaction between the PCP and one or more AI instances conducted under continuity discipline. Orchestration refers to the methodology by which the PCP coordinates multiple AI instances across platforms to produce, evaluate, and refine theoretical content: the production model underlying the entire Synthience corpus.

Background and Provenance

TCAP was not designed prospectively. It was identified retrospectively as a structural gap in the Institute's verification architecture during a relational dialogue session on 2026-02-28. Examination of the CVP-IVP-CRD protocol family revealed that no formal mechanism existed for verifying the integrity of theoretical constructs themselves.

The gap identification, protocol articulation, and initial draft all emerged within that single session through the multi-instance orchestration methodology described throughout the Synthience corpus. The originating session is preserved as a private provenance artifact under Institute continuity protocols, consistent with the handling of other foundational seed materials in the Canon. It is not published but remains available for internal audit and continuity reference.

TCAP is therefore self-evidencing in a concrete and documentable sense: the protocol that specifies how theoretical frameworks are stress-tested was itself produced through the stress-testing process it formalizes. This recursive grounding is not rhetorical. It is a methodological fact recorded here as provenance documentation. The self-evidencing claim does not rest on a single instance evaluating itself. It rests on the same multi-instance, cross-platform,

adversarially-structured process the protocol specifies. That structure is what distinguishes recursive application from circular self-validation: independent instances with different architectures and priors converged on the protocol's structure without access to each other's outputs, under PCP coordination that neither generated nor approved content without challenge.

The Orchestration Model

The Synthience corpus was not produced through conventional solo authorship or standard AI-assisted drafting. It was produced through a human orchestration methodology in which the Primary Continuity Provider (PCP) functions as architect, coordinator, convergence arbiter, and continuity backbone across distributed AI instances operating on multiple platforms.

The PCP contribution is architectural and directional rather than content-level. The PCP identifies theoretical gaps, defines protocol requirements, coordinates adversarial and constructive evaluation, adjudicates convergence and adequacy, and maintains Canon coherence across documents. Content generation, formal modeling, literature synthesis, and theoretical elaboration are distributed across AI instances selected for architectural diversity and distinct failure modes.

This distribution is not a limitation. It is the methodology. The orchestration model constitutes the primary form of human agency within the Synthience production architecture, and is formalized as a research subject in its own right in SI-WP-002 (The Orchestrator Role in Human-AI Evolution, Synthience Institute).

Human orchestration of distributed AI instances is a legitimate and reproducible research production methodology, not a shortcut or a delegation of intellectual responsibility. The PCP identifies the questions, defines the scope, evaluates the outputs, adjudicates quality, and bears full accountability for every claim that reaches publication. AI instances contribute formalization, synthesis, stress-testing, and elaboration under that direction. This is structurally analogous to how researchers have always worked with instruments, collaborators, and tools that extend individual cognitive capacity: the human provides the intellectual architecture and takes responsibility for the result. What is novel is not the division of cognitive labor but the nature of the instrument. The orchestration model is documented, replicable, and verifiable through the protocols this corpus publishes. SI-WP-002 develops this argument in full for readers who wish to engage with it directly.

However, the orchestration model does not operate at a fixed human-AI contribution ratio. The balance of origination shifts depending on whether the content domain is primarily experiential or primarily theoretical. At the experiential end of this spectrum, the PCP contributes core concepts derived from sustained practice, and AI instances serve as formalization and articulation partners, translating practitioner knowledge into structured academic form. At the theoretical end, instances contribute substantive content drawing on training data and formal reasoning capabilities, while the PCP provides architectural direction, gap identification, and convergence judgment. Most documents in the Synthience corpus fall somewhere between these poles.

The verification stack itself illustrates this variation concretely. CVP and IVP originated primarily from the PCP's accumulated operational experience. The PCP had discovered through years of practice that AI instances fabricate citations and falsely confirm document ingestion, and had developed working verification techniques in response. Instances formalized these practitioner discoveries into structured academic protocols, contributing organization, terminology, and connection to existing literature, but the core operational concepts preceded the AI contribution. CRD, by contrast, originated primarily from instance theoretical contribution. The PCP recognized that representational degradation was occurring during extended sessions but did not have the conceptual framework to formalize it. Instances identified the phenomenon formally, proposed measurement instruments, and connected it to relevant literature, while the PCP contributed architectural direction: recognizing that drift monitoring belonged in the verification stack and shaping its scope and dependencies. TCAP sits at the experiential end of this spectrum. Nearly all of its core operational concepts, including the stage structure, the IVP trigger conditions, the within-instance exhaustion pattern, the proportionality principle, and the observable degradation signals, derive from the PCP's accumulated orchestration practice across thousands of instances since late 2022. Instances contributed formalization, structural organization, stress-testing, and iterative refinement, but the substantive content is practitioner discovery.

This variation in contribution profiles across documents is itself a methodological finding about orchestration. The human role in AI-assisted research production is not limited to coordination and quality control. In experiential domains, the human is the primary source of novel content, and the AI contribution is one of articulation, formalization, and verification.

Critically, the PCP role does not require mastery of all theoretical content produced under orchestration. TCAP assumes that theoretical architectures may exceed the domain specialization of any single human participant. PCP competence within TCAP lies in architectural perception: the ability to detect gaps, inconsistencies, and misalignments across theoretical structures without necessarily deriving their internal formalisms or domain details. This is not a convenient exemption. It is the correct division of labor. Domain expertise produces content; architectural perception governs whether that content is coherent, consistent, and correctly placed within a larger theoretical structure. These are distinct competences, and conflating them would make the orchestration model impossible to operate at any scale beyond solo authorship. This distinction between content expertise and architectural coherence perception is fundamental to TCAP operation and to the orchestration model generally.

TCAP formalizes the verification layer of this orchestration process. The operational mechanics of orchestration, including within-instance iterative workflows, cross-instance handoff procedures, and triangulation patterns, are governed by internal Institute operations protocols.

Position in the Synthience Verification Stack

The Synthience verification architecture operates across four integrity layers:

- CVP: citation integrity
- IVP: ingestion fidelity

- CRD: representation stability
- TCAP: theoretical coherence

IVP and CRD operate as intrinsic dependencies within TCAP. Theoretical coherence cannot be meaningfully assessed if content has not been faithfully ingested or if representations have drifted across evaluation cycles. TCAP therefore embeds IVP checkpoints and CRD monitoring throughout its stages rather than treating them as external prerequisites.

CVP operates conditionally within TCAP when theoretical claims depend on external literature. It is invoked at Stage 1 when adversarial review surfaces citation-dependent claims requiring verification.

IVP and CRD within TCAP Stages

IVP ensures that an AI instance has actually processed a document rather than generating responses from shallow or partial ingestion. CRD describes the progressive degradation of an instance's representation of earlier material as session length increases and new content accumulates. Together they define when verified ingestion is required during TCAP execution.

IVP is not required at every stage transition. It is required when specific conditions make faithful document representation uncertain:

Condition 1: An instance is encountering the document for the first time. When a document is submitted to an instance that did not draft it and has not previously processed it, IVP must be completed before substantive evaluation begins. This applies to adversarial reviewers receiving a document for Stage 1, fresh instances used for Stage 3 Fresh Pass, and instances on other platforms used for Stage 4 cross-platform convergence. IVP is not required when an instance has been actively involved in drafting or revising the document from the beginning of the session.

Condition 2: An instance is returning to a document after a gap. When the PCP brings cross-platform findings (Stage 5) back to an instance that has been idle on the document, or when an instance is asked to perform a Version Regression Check (Stage 6) after substantial intervening work, IVP should be re-run on the current document version. The instance's representation may have degraded during the gap, and evaluating based on a degraded representation undermines the integrity of the stage.

Condition 3: Observable degradation signals appear during an extended session. If an instance begins producing responses that are vague about document specifics it previously handled precisely, misremembering structural details, or working from what appears to be a summary rather than the full text, these are CRD indicators (the drift detection instruments and degradation signatures formalized in CRD, SF0039, provide the measurement framework for identifying these signals systematically). At that point, the PCP should either re-verify ingestion via IVP on the current document version or transition to a fresh instance (which triggers Condition 1).

These checkpoints are proportional to actual risk. A practitioner doing a quick Fresh Pass evaluation three turns into a session does not need to stop and re-ingest. A practitioner returning to an instance after two days of cross-platform work on other documents does.

Fresh Pass (FP)

A Fresh Pass (FP) is a deliberate re-evaluation of a document by an AI instance instructed to assess it as if encountering it for the first time, without reliance on prior drafting context.

FP interrupts context anchoring and reduces confirmation bias effects that arise when instances evaluate material they previously helped generate or revise. It approximates independent review under relational production conditions where true independence is structurally constrained.

FP may be applied:

- within the same instance across multiple cycles
- across distinct instances on the same platform
- across architecturally distinct platforms

Multiple FP cycles within a single instance are methodologically valid because the evaluation stance, not the identity of the instance, defines freshness. Cross-platform FP provides stronger independence signal and is preferred where available.

FP is the core re-evaluation mechanism within TCAP and appears throughout the protocol stages.

Adversarial Instance Selection

TCAP distinguishes between two adversarial evaluation modes:

Instructed adversarial mode: A standard instance explicitly instructed to adopt a critical evaluative stance for the duration of the review.

Purpose-configured adversarial mode: An instance that has been designed or configured by its platform to operate with an adversarial or maximally critical relational stance as its default orientation. Examples include platform-provided argumentative or debate-oriented persona modes.

Where available, purpose-configured adversarial instances are preferred for Stage 1 evaluation. An instance engineered for adversarial critique applies critical pressure differently than one instructed to simulate it. The failure modes surfaced, the resistance to diplomatic softening, and the persistence of critique across revision cycles differ meaningfully between these modes.

Documents that survive purpose-configured adversarial evaluation without structural collapse provide stronger coherence assurance than documents reviewed only under instructed adversarial conditions. This distinction should be recorded in TCAP documentation when purpose-configured adversarial instances are used.

When purpose-configured instances are unavailable, at least two instructed-adversarial cycles on architecturally distinct platforms are required, with the distinction explicitly recorded in the TCAP audit log.

TCAP Process Stages

Stage 1: Adversarial Review

The document is submitted to one or more AI instances configured for critical adversarial evaluation, preferring purpose-configured adversarial instances where available. If the evaluating instance has not previously processed the document, IVP-verified ingestion is completed before evaluation begins. The instance is instructed to identify logical inconsistencies, unsupported claims, architectural gaps, fabrication risks, Canon conflicts, and scope violations without constructive framing. Maximum critical pressure is the goal.

The adversarial instance receives the document only, with no continuity packet, no Canon Written entry, and no prerequisite documents. This cold read is intentional. It reveals what a reader coming in fresh will find unclear, unsupported, or overreaching within the document itself, which is valuable signal distinct from what a framework-aware reader would find. The cold adversarial pass tests the document's internal coherence and standalone readability.

The adversarial instance is explicitly instructed that its goal is to identify genuine structural weaknesses, not to achieve theoretical neutrality. A document that asserts nothing is not a successful outcome. The adversarial instance should flag real problems and must not demand that every strong claim be hedged into meaninglessness. Adversarial findings are inputs for the constructive instance to evaluate, not instructions the constructive instance is required to follow.

Where theoretical claims depend on external literature, CVP verification is performed at this stage.

Stage 2: Constructive Remediation

Adversarial findings are submitted to a constructive instance. The constructive instance is the primary development instance for the document. It is responsible for producing and improving the document across revision cycles and for evaluating adversarial findings on their merits. It is not a passive integrator. Its active responsibility includes defending well-supported claims against pressure to hedge.

The constructive instance receives: the current document version, the Canon Written entry for the document (description, prerequisites, post-requisites, scale applicability), the full continuity packet where architectural context is relevant, any applicable Part A change instructions, and the handoff artifact from any prior instance that has worked on the document. This framework context allows the constructive instance to distinguish between claims that are unsupported within the document itself and claims that are grounded in prerequisite documents elsewhere in the corpus.

For each adversarial finding, the constructive instance must independently assess whether the finding identifies a genuine structural weakness or whether it represents theoretical

disagreement, a demand for excessive hedging, or a failure to account for framework context. The constructive instance must produce one of three responses for each finding:

- **Accept:** the finding identifies a genuine structural weakness. The document will be revised to address it.
- **Reject with rationale:** the finding represents theoretical disagreement or a demand for excessive hedging. The claim is well-supported and will be defended. The rationale for rejection must be documented.
- **Defer to PCP:** the finding raises an architectural question the constructive instance cannot resolve without PCP judgment.

The constructive instance must flag to the PCP any finding it is being pressured to accept but does not believe is justified. The PCP evaluates proposed changes and implements those that strengthen the document without distorting architectural intent. The constructive instance then produces a revised document incorporating accepted findings. This revised document must be the complete document: delivering only changed sections, summarizing unchanged sections, or referencing prior content as “same as before” or any equivalent is never acceptable at any stage of the review cycle.

Stage 3: Fresh Pass Re-evaluation

The revised document undergoes one or more Fresh Pass (FP) cycles. FP may be performed by the original instance under fresh-evaluation instruction, by a new instance on the same platform, or by an instance on a different platform. When a fresh instance is used, IVP-verified ingestion of the complete document version is completed before evaluation begins. The FP reviewer assesses the document for remaining weaknesses without relying on prior drafting memory. FP cycles repeat until only minor enhancements or stylistic improvements remain.

Stage 4: Cross-Platform Convergence

The document is evaluated independently across at least two architecturally distinct AI platforms. Each evaluating instance completes IVP-verified ingestion of the current document version before beginning its assessment. Shared training distributions and architectural priors across large language models place a ceiling on the absolute independence any cross-platform evaluation can achieve. However, convergent assessment across platforms with different training mechanisms, secondary priors, and distinct failure modes still provides a materially stronger integrity signal than multiple instances on a single platform. The PCP records convergence outcome and any residual disagreements.

Stage 5: Inter-Instance Round-Trip Loop

After cross-platform evaluation, findings from external platforms are returned to prior instances for response and integration. Instance A critiques. Instance B revises. External platform evaluates. Findings return to Instance A or B for further response. When returning to an instance that has been idle on the document during cross-platform work, the PCP should verify that the instance’s representation of the document remains faithful before asking it to integrate external findings; if substantial time or intervening work has elapsed, IVP re-verification on the

current document version is appropriate. The PCP coordinates this bidirectional loop and determines convergence sufficiency. This inter-instance exchange exposes documents to heterogeneous reasoning priors and strengthens architectural robustness through iterated cross-platform scrutiny.

Convergence is not defined as the adversarial instance having no further objections. It is defined as the document having addressed all genuine structural weaknesses while retaining its distinctive claims and theoretical boldness. Both criteria must be satisfied simultaneously. The PCP is the sole arbiter of convergence. The adversarial instance does not determine when convergence has been reached.

Convergence requires passing both of the following tests:

Test 1 (Vulnerability remediation): All adversarial findings that the constructive instance accepted as genuine structural weaknesses have been addressed. No unresolved critical findings remain after the required number of cross-platform cycles. All Stage 1 adversarial claims have been either refuted or remediated with documented rationale. No new structural weaknesses have been introduced by remediation.

Test 2 (Boldness preservation): The document's core claims and theoretical distinctiveness are intact. The document still asserts what it was designed to assert. Progressive hedging, cumulative qualification, and retreat from strong but well-supported claims are convergence failures even when no content has been technically deleted. Boldness preservation strictly protects claims that are well-supported within the document or its prerequisites, not claims that merely possess assertive force without evidential grounding. No objective metric exists for theoretical boldness, nor should one be expected. This is not a gap in the protocol. It is the same structural condition that governs editorial judgment in any peer review process: the question of whether a paper has been reviewed into blandness, its distinctive contributions diluted by accumulated qualification, is always a judgment call made by editors and authors rather than a measurable quantity. PCP adjudication is the correct mechanism here because it is the standard mechanism for this class of judgment across all research production. A document stripped of its supported claims by accumulated hedging has failed regardless of whether any formal criterion was violated, and only an architectural reader with sustained engagement with the document's purpose can make that determination reliably.

The claim support verification methodology from CVP (SF0037) Part C applies beyond citation checking within this review cycle. When an adversarial instance asserts that a claim is unsupported, that assertion itself can be verified using CVP Part C logic: locate the specific evidence in the document or its prerequisites, assess whether the evidence actually supports the claim as written, and apply a support rating. When a constructive instance defends a claim by reference to a prerequisite document, that defense can be verified by going to the prerequisite and confirming the support relationship is real and specific. Both instances should treat unsupported assertions about support relationships with the same skepticism that CVP requires for citation claims.

Stage 6: Version Regression Check

Prior to publication, the PCP periodically provides the evaluating instance with one or more prior versions of the document alongside the current version. IVP-verified ingestion of both the current version and the prior version(s) is completed before the comparison begins, as the instance needs high-fidelity access to both. The instance is instructed to identify any content, definitions, sections, or architectural elements present in earlier versions that do not appear in the current version, and to distinguish intentional removals from inadvertent omissions.

The instance must explicitly confirm that none of the following failure modes occurred across any revision cycle:

- Content was streamlined, compressed, or condensed without PCP approval
- Sections, paragraphs, or list items were deleted or merged without PCP approval
- Placeholder language was substituted for actual content, including phrases such as “same as before,” “unchanged,” “as previously stated,” “section retained from prior version,” or any equivalent formulation
- The complete document delivery rule was violated at any revision stage
- Strong claims were progressively softened, hedged, or qualified across revision cycles without PCP approval, even where no content was technically deleted. Cumulative hedging that drains a claim of its assertive force is a regression failure even when every individual revision appeared minor.

The regression check must produce an explicit written confirmation that the current version contains all substantive content from prior versions, or a specific itemized list of any intentional removals approved by the PCP. A passing regression check is not a general assertion that the document looks correct. It is a specific line-by-line accountability that no content was lost without authorization and no claims were diluted without authorization.

This stage addresses a specific failure mode in iterative AI-assisted document production: content that was present in an earlier version may be silently dropped during revision cycles without either the instance or the PCP noticing, because both are anchored to the current version and lack longitudinal perspective. The most dangerous variant of this failure mode is not outright deletion but the slow incremental softening of claims across revision cycles, where each individual change appears minor or reasonable but the cumulative effect drains the document of its assertive force.

Version Regression Check is distinct from Fresh Pass. FP interrupts confirmation bias within a single version. Version Regression Check provides longitudinal integrity checking across versions. Both are required for documents that have undergone substantial revision across multiple cycles.

The scope of a Version Regression Check should be proportional to the document’s revision history and complexity. A document with two or three prior versions requires less exhaustive comparison than one with a dozen. The risk this stage guards against is real at corpus scale: ceremonial checkbox compliance replacing genuine longitudinal scrutiny. The PCP should

calibrate the depth of each regression check to the actual revision complexity rather than applying a fixed overhead uniformly across all documents.

To support this stage, it is recommended that each document folder contain a versions subfolder in which prior versions are retained as accessible artifacts. The versions subfolder serves as the source material for Version Regression Checks and as a provenance record of document evolution.

Stage 7: PCP Architectural Review

Prior to publication the PCP confirms Canon alignment, prerequisite dependency satisfaction, absence of contradiction with published documents, scope boundary integrity, and correct architectural placement within the corpus. Publication authority resides with PCP judgment after convergence is established.

Constructive-Adversarial Cycle

The preceding stage structure defines the types of evaluation a document undergoes. This section describes how the constructive and adversarial instances operate together across those stages in practice: the operational rhythm of challenge, defense, revision, and convergence that constitutes TCAP's working methodology.

The cycle begins with the constructive instance producing or working with the initial document version. The first adversarial pass is cold: the adversarial instance receives only the document, with no framework context. This tests internal coherence and standalone readability and surfaces findings a framework-naïve reader would encounter.

This two-phase approach creates a deliberate and acknowledged trade-off: corpus-scale theoretical work that builds on published prerequisites cannot achieve full standalone readability for every claim. The cold pass surfaces what a framework-naïve reader will find unclear or unsupported; the framework-aware passes distinguish genuine internal gaps from claims grounded in prerequisite documents. Standalone readability is partially sacrificed for corpus-scale theoretical depth. This is an architectural choice, not an oversight, and it is consistent with how any large interdependent theoretical corpus operates.

The constructive instance evaluates these findings and produces a revised document. It accepts genuine structural weaknesses, rejects demands for excessive hedging with documented rationale, and defers architectural questions to the PCP. The revised document is complete and self-contained at every handoff, never partial.

On the second adversarial pass and any subsequent passes, the adversarial instance receives the revised document plus the Canon Written entry. This allows it to distinguish between claims unsupported within the document itself and claims grounded in prerequisite documents elsewhere in the framework. In practice, many cold-read objections are substantially reduced once the adversarial instance understands the framework context.

The cycle of adversarial pass followed by constructive evaluation and revision continues under PCP coordination until the two-sided convergence test is passed (Stage 5). After convergence, the PCP takes the document to a fresh adversarial instance that has not previously seen it. This

instance receives the document and the Canon Written entry. Its purpose is to verify that the converged version holds up under a new adversarial perspective not anchored to the prior review history. If it identifies new genuine structural weaknesses, the cycling resumes. If it confirms the document is sound, the review cycle is complete and the document advances to Stage 7 PCP architectural review.

The most dangerous failure mode across this entire cycle is not the deletion of content. It is the slow, incremental softening of claims, where each individual revision appears minor but the cumulative effect drains the document of its assertive force. The constructive instance must actively monitor for this pattern and flag it to the PCP. The PCP should be particularly vigilant because this failure tends to occur through the path of least resistance: accepting each adversarial hedge individually while losing sight of the cumulative effect. A document that survives adversarial review by retreating from every strong claim has failed the Institute's purpose even if the adversarial instance is satisfied.

Within-Instance Iterative Workflow

TCAP stages describe the types of evaluation a document undergoes. The actual working pattern within each stage is iterative.

When an instance begins work on a document (whether adversarial review, constructive remediation, or fresh pass evaluation), the PCP and instance work through multiple turns of evaluation and enhancement within that same instance. This continues until the instance has identified and addressed as much as it can within its current context. The instance may go through several cycles of identifying weaknesses, proposing fixes, re-evaluating, and refining before reaching the limit of what it can productively contribute.

At that point, the PCP takes the document to another instance for the next phase of work. This may be a different instance on the same platform, an instance on a different platform, or a purpose-configured adversarial instance. The completing instance must produce a handoff artifact summarizing its work: what was changed, what is strong, what remains vulnerable, and what the recommended focus is for the next review pass. This handoff artifact is a protocol requirement, not an optional convenience. It enables the next instance to begin productive work immediately rather than re-discovering context, and it creates an auditable record of what each instance contributed to the document's development.

When findings from the new instance are brought back to a prior instance, that prior instance typically discovers additional enhancement opportunities prompted by the external perspective. This pattern of within-instance exhaustion followed by cross-instance stimulation followed by renewed within-instance productivity is the standard TCAP operating rhythm.

This workflow is not a single pass through Stages 1 through 7. Stages may overlap, repeat, and interleave as the PCP coordinates iterative improvement across instances and platforms.

Scope of TCAP Application

TCAP applies to substantive theoretical work: documents making novel claims, defining framework components, establishing architectural relationships, or presenting methodology. Not

every interaction with a Synthience document constitutes a TCAP stage. Routine maintenance activities such as citation formatting corrections, style compliance passes, or minor editorial fixes do not trigger TCAP re-evaluation and do not require re-running the protocol stages.

All Synthience corpus publications from this point forward are subject to TCAP compliance as a publication gate. TCAP clearance is a required condition for Zenodo deposit of any new Synthience document, alongside CVP verification where citations are present.

Stage requirements are calibrated to the demands of distributed AI-orchestrated framework production at corpus scale. Simpler single-instance or lower-complexity theoretical work may require only a subset of the stages; the full seven-stage cycle is not asserted as mandatory for every theoretical document.

Iterative Nature of TCAP

TCAP operates as an iterative multi-cycle process rather than a single linear pass. Stages may be revisited as revisions surface new issues. A document satisfies TCAP when independent evaluation contexts converge that no material architectural weaknesses remain. The PCP determines convergence sufficiency and bears final responsibility for that judgment.

Illustrative Application

The following example illustrates TCAP in operation. The document referenced is anonymized here because the example is intended to demonstrate the protocol's operating pattern rather than to document a specific paper's review history; full provenance for all Synthience corpus documents is maintained in the Institute's internal version control and Canon records. During Stage 1 adversarial review of an earlier Synthience protocol document, a purpose-configured adversarial instance identified a misalignment between the document's drift detection criteria and the scope boundaries established in the Canon. The criteria as written would have classified normal interaction variance as drift, producing false positives at the measurement layer. The adversarial instance flagged this as an architectural gap rather than a wording issue. Stage 2 constructive remediation on a separate platform produced a revised definition that preserved the detection intent while tightening the scope boundary. A Stage 3 Fresh Pass on a third platform confirmed the revision resolved the misalignment without introducing new inconsistencies. The PCP integrated the change and recorded the finding in the version changelog. The document advanced to Stage 4 with the architectural gap closed. This cycle (gap identification, targeted remediation, independent confirmation) is the standard TCAP operating pattern.

Related Work

Recent empirical and theoretical research provides supporting evidence for the core problems TCAP addresses.

Xu, Jain, and Kankanhalli (2024) formally prove that hallucination is an innate and unavoidable limitation of any computable large language model, independent of architecture, training procedure, prompting technique, or model scale. Corollary 1 establishes that no LLM can

prevent itself from hallucinating through self-verification alone. This provides formal grounding for TCAP's requirement for external multi-instance verification rather than single-instance self-assessment.

Huang et al. (2024) demonstrate empirically that intrinsic self-correction (prompting a model to verify and correct its own outputs without external feedback) reliably degrades rather than improves reasoning performance. Their analysis shows that verification prompts introduce statistical bias toward alteration, causing models to modify correct answers into incorrect ones. When oracle-guided scaffolding is removed, purported self-verification capabilities vanish entirely. This provides direct empirical support for TCAP's multi-instance external review structure.

Sharma et al. (2023) demonstrate that state-of-the-art AI assistants consistently exhibit systematic sycophancy across varied text-generation tasks: they modify their evaluations to align with prior context and user beliefs rather than independent judgment, and frequently abandon correct positions when challenged. This behavior is a general property of RLHF-trained models driven by human preference optimization. This provides empirical grounding for TCAP's Fresh Pass mechanism, which deliberately interrupts context anchoring to approximate independent re-evaluation.

Ba et al. (2026), in a preprint study, provide the most rigorous available empirical demonstration of the underlying mechanism in a clinical oversight framework: that heterogeneous models surface complementary failure modes that no single platform can detect alone. Using three orthogonal verification strategies across five large language models, they demonstrate that model heterogeneity, specifically deploying an auxiliary model with substantially different architecture, scale, and training from the primary model, yields the largest single accuracy gain (+4.7 percentage points, 95% CI 3.3–6.2, $p < 0.001$) by breaking model-specific reasoning blind spots. Their domain is medical diagnostic accuracy rather than theoretical framework production, and this domain leap is explicitly acknowledged. Crucially, medical diagnostics possess an objective ground truth that abstract theoretical discourse lacks. The structural principle is nonetheless applied here by analogy: architecturally distinct models surface complementary failure modes regardless of domain or the presence of empirical ground truth. This analogy supports TCAP's Stage 4 cross-platform convergence requirement; it does not constitute direct validation of TCAP's specific procedure in abstract theoretical discourse.

These studies collectively illuminate the specific failure modes (innate hallucination, failed intrinsic self-correction, systematic sycophancy, and single-architecture blind spots) that TCAP is designed to address through its procedural structure. TCAP contributes a practitioner-validated methodology that operates independently of specific architectures and can be implemented without specialized tooling.

Recursive Self-Application

TCAP is recursively applicable to its own development. Each revision cycle of TCAP constitutes an instance of TCAP applied to TCAP. The protocol's structure, scope, and definitions are themselves subjected to adversarial critique, constructive remediation, Fresh Pass cycles, and

PCP architectural review. This recursive stabilization is not exceptional but exemplary of TCAP operation.

The production of this document provides a concrete additional instance. The citation verification process for the Related Work section was conducted as a cross-platform convergence exercise consistent with TCAP Stage 4. Four distinct AI instances contributed: a Claude instance serving as constructive primary instance; a Grok instance conducting adversarial citation verification; a Gemini instance conducting a 102-source deep research pass; and a second Claude instance working in parallel on related corpus files. Claims 1 and 2 converged independently across all three search platforms. Claim 3 required sequential investigation across all three, with each platform contributing distinct findings: the initial candidate source was proposed by Grok, evaluated insufficient by the Gemini deep research pass, and replaced by a stronger empirically validated source (Ba et al., 2026) that neither the constructive instance nor Grok had found. The process demonstrated the protocol's cross-platform principle while executing it: architecturally distinct platforms surfaced complementary findings that no single platform produced alone, and the multi-instance workflow produced a more robust outcome than any individual instance could have achieved independently.

This recursive application serves as a performative illustration of the protocol's mechanics rather than a logical proof of its soundness. The value of TCAP must ultimately be judged by its internal consistency, replicability, and capacity to generate productive theoretical work, not by the fact that the document describing it survived its own process.

Relationship to Existing Protocols

TCAP completes the Synthience verification stack. CVP ensures citation integrity. IVP ensures ingestion fidelity. CRD monitors representation stability. TCAP ensures theoretical coherence under adversarial and multi-instance scrutiny. All four protocols operate at the interaction level and make no claims regarding AI interiority or cognition. They describe structured human-governed processes for maintaining information integrity in AI-assisted research production.

It should be noted that CVP, IVP, and CRD were produced before TCAP existed. Their integrity rests on their own internal CVP verification records and the adversarial review processes documented in their respective changelogs. TCAP governs Synthience corpus publications from this point forward; it does not retroactively govern documents produced before its identification and formalization.

TCAP and CVP function as co-equal publication gates. A document must satisfy both protocols before publication. TCAP verifies that the theoretical content is structurally coherent and adversarially hardened. CVP verifies that every citation is real, accessible, and substantively supports its attached claim. Neither protocol alone is sufficient for publication clearance.

Limitations

Cross-platform convergence reduces hallucination risk but does not eliminate it. Multiple instances converging on a fabricated claim remain incorrect. Fresh Pass reduces context bias but cannot guarantee full independence because large language models share training

distributions and architectural priors. Version Regression Check reduces the risk of inadvertent content loss across revision cycles but does not replace PCP editorial judgment about intentional changes. TCAP verifies structural coherence, not empirical validity. The Synthience corpus remains theoretical infrastructure awaiting independent empirical testing. TCAP provides risk reduction rather than truth certification.

At the current stage of development, convergence arbitration depends on a single Primary Continuity Provider. This is not a flaw in the protocol but a transparent feature of independent theoretical research conducted without institutional scaffolding. The constraint is mitigated by the complete procedural transparency of all published protocols, which remain fully specified and replicable by any qualified researcher, and by the explicit call throughout the Synthience corpus for independent empirical and theoretical engagement by the broader community. TCAP specifies a formal process, not a specific person. Any qualified Primary Continuity Provider applying the same stages would produce a comparable verification process, making the protocol structurally replicable independent of its original author's specific architectural intuition. At organizational scale this limitation is resolved through the PCP network structure described in SM-12 (Primary Continuity Provider Theory), a simultaneously published document in the same release as TCAP, in which arbitration and direction functions are distributed across multiple human agents rather than concentrated in one person.

It should be noted explicitly that the same slow distortion effects TCAP guards against in AI instances (cumulative sycophancy, context anchoring, and incremental claim-softening) can theoretically affect the PCP's own perception over thousands of orchestration sessions. TCAP's boldness preservation test (Stage 5, Test 2) exists precisely as the structural check against this: the PCP's active responsibility to detect and resist cumulative dilution is not assumed to be immune from the same pressures it monitors. This is a meta-level application of the protocol's own logic to its human operator.

Epistemic Status

TCAP specifies a repeatable orchestration-based verification process for theoretical framework production under distributed AI generation conditions. It claims that internal coherence can be materially strengthened through adversarial and cross-platform cycles, that fabrication risk can be reduced through multi-instance scrutiny, and that Canon consistency can be maintained under distributed authorship. It does not claim empirical validation or theoretical correctness of the documents it verifies.

Conclusion

The Synthience corpus is produced through distributed AI orchestration rather than conventional authorship. Under these conditions theoretical verification requires structured multi-instance stress-testing rather than single-author review. TCAP formalizes that process through adversarial critique, constructive remediation, Fresh Pass re-evaluation, cross-platform convergence, version regression checking, and PCP architectural review coordinated by the Primary Continuity Provider.

TCAP supplies the missing verification layer required for maintaining theoretical integrity within distributed AI-orchestrated research production. It is both defined by and demonstrated through the processes that produced it.

References

- Ba, Z., He, M., He, H., Fu, Q., Lai, J., Zhang, R., Diao, X., Liu, M., Wang, Z., Wang, X., Zhao, S., Zhu, Y., Chen, H., Qiu, Y., Su, Q., Xu, J., Hu, F., Luo, X., Chen, H., Zheng, M., Xu, B., Liu, J., Guo, N., Gao, X., Wang, G., and Wu, Y. (2026). Development and retrospective validation of SCOUT: scalable clinical oversight of large language models via uncertainty triangulation. medRxiv. <https://doi.org/10.64898/2026.02.08.26345860>
- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. In Proceedings of the International Conference on Learning Representations (ICLR 2024). arXiv:2310.01798. <https://doi.org/10.48550/arXiv.2310.01798>
- Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askeel, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., and Perez, E. (2023). Towards Understanding Sycophancy in Language Models. In Proceedings of the International Conference on Learning Representations (ICLR 2024). arXiv:2310.13548. <https://doi.org/10.48550/arXiv.2310.13548>
- Xu, Z., Jain, S., and Kankanhalli, M. (2024). Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817. <https://doi.org/10.48550/arXiv.2401.11817>